Final Project RIVAL

Agwyn Satria Diaurahman

PROJECT OVERVIEW

- Modifying an existing ROS2 x Gazebo Workshop
- Completing all 3 of the following Missions
  - Mission 1: TeleOp Control
  - Mission 2: Go to center (0, 0) and wait for 3s
  - Mission 3: Scan and save aruco data (manually/automated)
- Earn Bonus Points (if possible)

Main Architechture

Waypoint_controller

Controls the overall rover behaviour and mission control

Subscribes:
- localization/pose

Publishes:
- mission3/logging_enabled
- mission3/scanning_enabled
- navigation/waypoint_route

position_logger

Operating datas colllected from imu and odometer

Subscribes:
- /imu
- /odom

Publishes:
- localization/pose

aruco_detector

Detect, save, and open window for aruco

Subscribes:
- /mission3/scanning_enabled
- /mission3/logging_enabled
- /camera/image_raw
- /camera/camera_info

Publishes:
- None

driving_node

Actually drives the rover to a designated position

Subscribes:
- /navigation/target

Publishes:
- /cmd_vel (to bridge)
- /navigation/arrived

navigation_node

Finds the shortest route from waypoint A to B

Subscribes:
- /navigation/waypoint_route
- /localization/pose

Publishes:
- /navigation/target

How to Run

Langsung jalankan

```
chmod +x runbuild.sh
./runbuild.sh
```

waypointController

+ Melihat daftar waypoint

```
show waypoints
```

+ Melihat daftar AdjList

```
show graph
```

+ Sintaks moveto

```
moveto <waypoint 1> <waypoint 2> ....
```

Contoh

```
moveto 1 10 --read-aruco-logged
```

Argumen tambahan

--read-aruco-logged
--read-aruco-non-logged

How to Run

Teleop

Buka 2 terminal, masing masing menjalankan

```
source install/local_setup.bash
ros2 launch sim_bringup dummy.launch.py
```

```
chmod +x teleop.sh
./teleop.sh
```

Dissection

Poslogger

[] Node Variables

- self.x
- self.y
- self.z
- self.orientation
- self.previous_time
- self.imu_subscription
- self.odom_subscription
- self.pose_publisher

[] Node Methods

1. **imu_callback(self, msg)** → mengambil data imu dan menormalisasikannya
2. **publish_pose(self, stamp)** → mempublish ke publisher pose_publisher
3. **odom_callback(self, msg)** → mengambil data dari odometer, mengolahnya dengan data imu (orientasi), serta memanggil publish_pose()

DrivingNode

[] Node Variables

- self.x, y, yaw
- self.goalX, goalY
- self.arrived
- self.cmd_publisher
- self.pose_subscription
- self.status_publisher
- self.timer

[] Node Methods

1. **pose_callback(self, msg)** → mentranslate pose yang didapat dari subscriber menjadi x, y, dan yaw yang lebih mudah diolah
2. **set_target(self, x, y)** → mengupdate untuk menuju {x, y}
3. **target_callback(self, msg)** → terus mengecek apakah target tujuan masih valid
4. **control_loop(self)** → loop untuk terus terusan mengecek target dan memberi perintah untuk jalan ke target, juga mengupdate target bila ada perubahan target

Dissection

NavigationNode

[] Node Variables

- self.target_publisher
- self.route_subscription
- self.arrival_subscription
- self.pose_subscription
- self.mission_status_publisher
- self.x, y
- self.route
- self.current_index
- self.requested_route, requested_index
- self.final_target
- self.final_target_pending
- self.waiting
- self.current_target_is_final

[] Node Methods

1. **set_mission_active(self, active)** → mencatat bahwa ada misi aktif, bakalan nyupir
2. **route_callback(self, msg)** → menerima list waypoint melalui msg, memeriksa apakah localisasi ada
3. **pose_callback(self, msg)** → yeaa ngambil pose dari rover si
4. **destination_callback(self, msg)** → mengecek kita sedang di mana, dan akan merancang bakal lewat rute mana aja
5. **find_destination(self, x, y)** → mengecek waypoint mana aja yang terdekat dengan sekarang
6. **send_target(self, x, y)** → ngepublish target untuk drivingNode
7. **start_requested_destination(self)** → memeriksa apakah semua goal sudah dicapai, kaya bis surabaya-mojokerto-sidoarjo
8. **plan_route(self, x, y)** → mirip no 7 tapi dari titik random
9. **send_next_target(self)** → mengirim target selanjutnya kalau sekarang udah kecapaian
10. **arrival_callback(self, msg)** → udah nyampe belum?

Dissection

ArucoDetector

[] Node Variables

self semua
- bridge
- aruco_dict
- parameters
- camera_matrix
- dist_coeffs
- captured_ids
- detection_counts
- required_detection_frames
- storage_path
- image_path
- metadata_path
- latest_frame
- latest_ids
- latest_corners
- logging_requested
- mission_active
- log_directory
- scannng_enabled
- scanning_subscription
- logging_subscription
- mission_subscription
- detection_log
- capture_service
- image_sub
- info_sub

[] Node Methods

1. **info_callback(self, msg)** → nerima input informasi kamera yh buat kalibrasi
2. **logging_callback(self, msg)** → saklar buat logging
3. **mission_callback(self, msg)** → “Is it a scanning mission?”
4. **image_callback(self, msg)** → ngambil image dari bridge, convert ke grayscale, detec corner, output, ngecek apakah double counting id, ngecek sekarang lagi ngedetect atau engga, nggambar marker untuk aruco, dan capture otomatis kalau semua parameter terpenuhi dan mencegah double counting dalam misi yang sama.
5. **scanning_callback(self, msg)** → saklar buat scanning
6. **save_capture(self)** → menyimpan gambar untuk deteksi otomatis, sekalian metadata dengan json nya yang berisi id aruco
7. **capture_callback(self, request, response)** → callback buat ros
8. **log_detections(self, ids, corners)** → kita ngelog

Evaluation

Pros

- Odom dan IMU bekerja dengan cukup stabil
- Autonomous capture aruco bekerja dengan sangat baik

Cons

- Kecepatan rover masih cukup lambat
- Belum memanfaatkan LIDAR
- fitur WaypointController masih bergantung kepada perintah operator



TerimaKasih